MoonPark Pricing Engine

A secure, extensible API for versioned parking fees

Correct pricing.
Explicit tradeoffs.

Java 25 | Spring Boot 3.5

PostgreSQL | Redis | Bucket4j | Caffeine

01 DOMAIN

Pricing rules and versioning

02 PLATFORM

Identity, quotas and resilience

20 MINUTES / Architecture > request flows > security > delivery > discussion

One pricing API. Three different charging models.

Business correctness comes before infrastructure complexity.

M1 / STARTED MINUTE

60 NOK per hour

Charged per started minute

61 minutes = 61 NOK

M2 / STARTED HOUR

Weekday: 100 NOK/hour

Weekend: 200 NOK/hour

Charge each started hour

M3 / TIME WINDOWS

Mon-Sat, 08:00-16:00:

First 60 daytime minutes free;

then 2 NOK/started minute.

Outside daytime: 3 NOK/min.

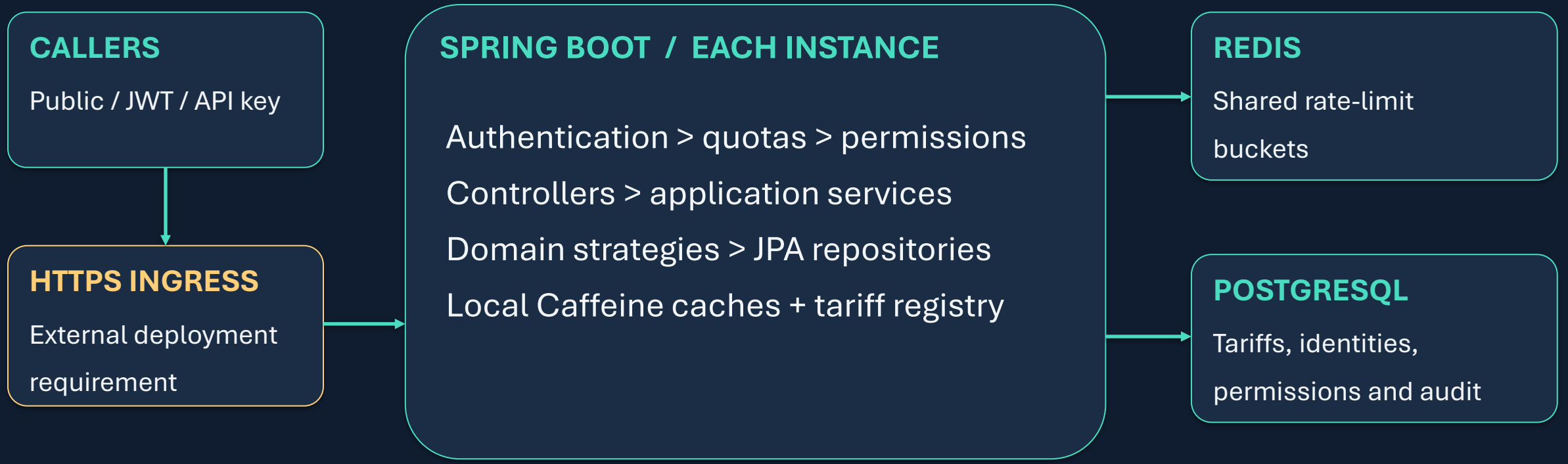
Sunday: free.

Core rule: select the tariff effective at the START of the parking session.

Sources: README.md; model/domain/tariff

A layered service with shared infrastructure

One deployable application; independent instances can serve the same API.



Redis is NOT the fee cache. Metrics, logs and tracing integrate with external tooling.

Sources: SecurityConfig; FeeCalculationService; CachedTariffRegistry; application.yml

Separate policy from transport and persistence

Pricing algorithms remain isolated from Spring MVC and database access.

CONTROLLERS	HTTP contracts, DTO validation and endpoint permissions
APPLICATION SERVICES	Use cases, transactions, cache coordination and audit
DOMAIN STRATEGIES	Tariff interface + three charging implementations
REPOSITORIES	JPA persistence; Liquibase owns schema evolution

New zone + existing rule = configuration. New charging algorithm = strategy + factory wiring.

Sources: controller; service; dao; model/domain/tariff; TariffFactory

The normal quote path avoids a database read

Validate the period, select the effective tariff, calculate and expose its version.

01 REQUEST
GET /api/v1/takst
zone + start + end

→

02 CACHE
Caffeine lookup
Miss enters calculation

→

03 DOMAIN
Validate > tariff
> pricing strategy

→

04 RESPONSE
Fee result
+ tariff version

TIME SEMANTICS

Default timezone: Europe/Oslo; max period: 31 days.

EXAMPLE

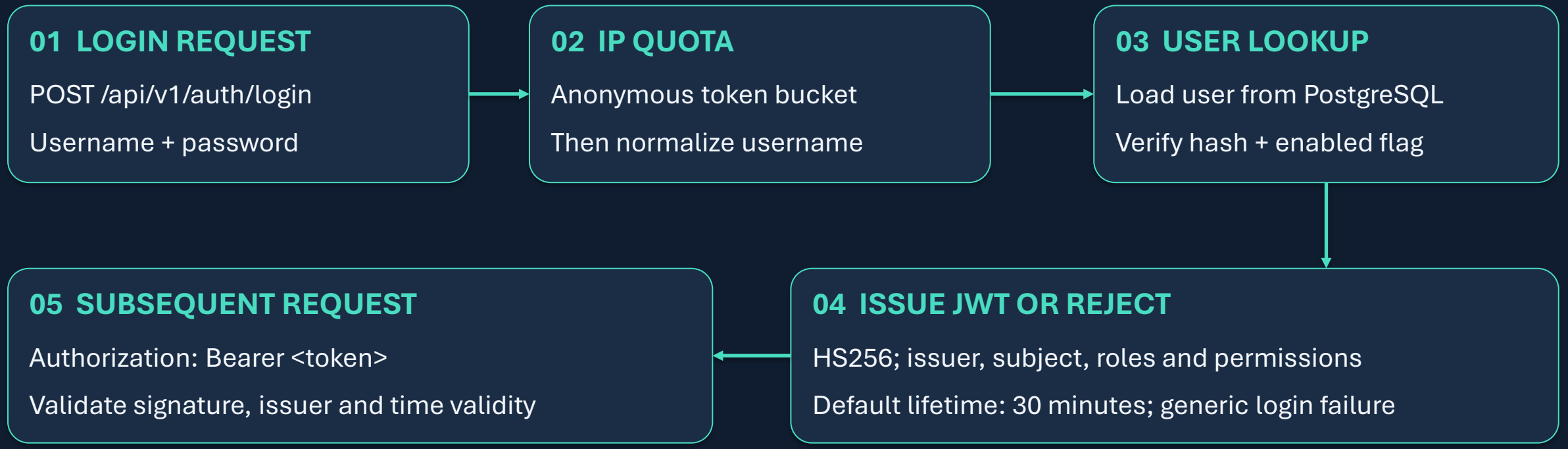
M1 | 22 Oct 2026, 08:00-09:01 | 61 NOK

Boundary cases: started units, midnight, weekends, daylight saving and effective dates.

Sources: TariffController; FeeCalculationService; CachedTariffRegistry; application.yml

Login: credentials in, short-lived access token out

Login is public, but anonymous requests consume an IP-based quota.



Unknown user? Still verify a dummy hash to reduce user-enumeration timing signals.

Sources: AuthService; TokenService; JwtConfig; SecurityConfig; RateLimitFilter

Different identities. One permission model.

Stateless authentication establishes identity; permissions authorize the operation.

HUMAN USERS / JWT

Password hashing: bcrypt by default

Roles + permissions carried in JWT

Method-level @PreAuthorize checks

Existing tokens keep old claims until expiry

PARTNER SYSTEMS / API KEY

256 random bits; SHA-256 hash stored

Raw key shown once at creation/rotation

Check enabled flag and expiry on lookup

Local key cache: up to 60s propagation

Send JWT OR X-API-Key. Sending both credentials is rejected with HTTP 401.

Sources: ApiKeys; ApiClientService; SecurityConfig; controller @PreAuthorize annotations

Identify > price the request > consume tokens

Bucket4j token buckets protect callers fairly across application instances.



Health and API-documentation routes bypass this limiter. Authentication runs before it.

Sources: RateLimitFilter; RateLimitPlan; RedisRateLimitStore; LocalRateLimitStore

Sustained rate is not the same as burst capacity

Greedy refill replenishes tokens continuously; capacity limits instantaneous bursts.

Plan	Refill / minute	Burst tokens	Typical identity
ANONYMOUS	10	10	Remote IP
FREE	60	20	API client
STANDARD	600	100	JWT user / API client
PARTNER	6,000	500	API client
INTERNAL	6,000	500	JWT admin

Redis failure: local quotas for 30s, then retry Redis. Availability wins; global limits weaken.

Sources: RateLimitPlan; Buckets; RedisRateLimitStore; application.yml

Defense in depth, with explicit trust boundaries

Application safeguards complement deployment controls; neither replaces the other.

IMPLEMENTED IN THE APPLICATION

- Least privilege: roles + fine-grained permissions
- Hashed passwords and API keys
- Generic login errors + dummy hash check
- Validation, quotas and concurrency limits
- Audit events + content-security policy

DEPLOYMENT RESPONSIBILITIES

- HTTPS at a trusted ingress / reverse proxy
- Sanitize forwarded headers; block bypass paths
- Private database and authenticated Redis
- Inject production secrets; remove bootstrap secret
- Restrict monitoring and rotate credentials

CSRF is disabled for header-based stateless auth; revisit if browser-cookie authentication is added.

Sources: SecurityConfig; AuthService; ApiKeys; AuditService; README.md

Versioned tariffs, local caches, bounded propagation

PostgreSQL is authoritative; instances refresh their in-memory view independently.

ADMIN CHANGE

Commit tariff transaction

Publish TariffChangedEvent

→

LOCAL INSTANCE

After-commit refresh

Clear fee cache on change

→

OTHER INSTANCES

Scheduled refresh: 60s

Observe changed versions

FEE CACHE

Caffeine: up to 100,000 entries; 10-minute TTL.

Each instance owns its cache.

FAILURE BEHAVIOR

Failed refresh retains last loaded tariffs.

Cross-instance consistency is eventual.

No instantaneous global invalidation: another instance may serve old results until it refreshes.

Sources: CachedTariffRegistry; TariffRefreshScheduler; FeeCalculationService; application.yml

Resilience is a policy, not just a retry

Different failures intentionally produce different availability behavior.

Concern	Current behavior
Calculation overload	Bulkhead: 200 concurrent calls; max wait 50ms
Redis unavailable	Per-instance fallback; Redis excluded from readiness
Database unavailable	Readiness can fail, even with tariffs retained in memory
Observability	Structured logs, Prometheus metrics, OTLP tracing support

Liveness asks: restart? Readiness asks: receive traffic? External dashboards are not bundled.

Sources: application.yml; application-prod.yml; TariffsHealthIndicator; logback-spring.xml

Test business boundaries; package reproducibly

Domain tests are fast. Full-context integration tests use disposable PostgreSQL.

QUALITY CONTROLS

Tariff, validation and service tests

MVC security tests for public fee routes

Testcontainers integration tests

90% domain-package coverage gate configured

Docker unavailable? Integration tests may skip.

DELIVERY PATH

Gradle > executable application JAR

Java 25 runtime; non-root Docker image

Liquibase migrations; JPA schema validation

Production environment supplies secrets

Post-deploy readiness and smoke checks

Quality gates and capacity limits are configuration, not claims of passing tests or measured throughput.

Sources: build.gradle; src/test; Dockerfile; README.md

A one-minute tour of the API contract

Use Swagger or prepared requests; keep credentials and production data off-screen.

Step	Action	Expected result
01	GET /api/v1/takst | M1, 08:00-09:01	61 NOK with seeded tariff
02	POST /api/v1/auth/login	Access token + token lifetime
03	Call a permitted protected endpoint	Authorized response
04	Call without the required permission	403 Forbidden
05	Exhaust an anonymous bucket in local dev	429 + Retry-After

Example date: 22 Oct 2026. Quota demos depend on remaining tokens and refill; rehearse beforehand.

Sources: README.md; TariffController; SecurityConfig; RateLimitFilter

Know the tradeoffs. Be clear about the next step.

A focused design is more credible than claiming every production concern is solved.

Current decision	Tradeoff / next improvement
Locally issued HS256 JWTs	Shared-secret lifecycle; consider external identity provider
Permissions embedded in tokens	Changes wait for expiry; add revocation if required
Local key and tariff caches	Propagation delay; consider cross-instance invalidation
Limiter follows authentication	Invalid credentials may bypass it; add edge throttling
Custom standalone Redis client	TLS / cluster support needs further implementation

DISCUSSION / Why a layered service? What fails first? What would you measure before scaling?

Sources: JwtConfig; ApiClientService; RedisRateLimitStore; README.md